readcon-core: a reader and writer for the CON checkpoint formatreadcon-core: a reader and writer for the CON checkpoint format

Rohit Goswami[a]

[a]*Science Institute and Faculty of Physical Sciences, University of Iceland, Reykjavík, Iceland*

Abstract

Rare-event methods store an atomic geometry many times while they search for a saddle or a path. The eOn family writes that checkpoint as CON text. A restart needs a per-axis constraint, the atom order from before type grouping, and a velocity time unit. Specification 3.1 states that contract. readcon-core 0.16.2 is a Rust reader and writer for CON versions 1, 2, and 3, with a C ABI and bindings for C++, Python, Julia, and Fortran. This paper summarises the specification and records where the 0.16.2 code does not yet implement a clause. The public HCN to HNC endpoints of an eOn nudged elastic band workflow are rewritten as CON 3.1. readcon-core returns the mask, the id, and the velocity numbers from that file and stores the time-unit string. The ASE eon reader does not keep the mask or the id. Paired timings against ASE, a chemfiles XYZ read, MDAnalysis H5MD, and h5py are reported with bootstrap intervals.

Keywords: CON, checkpoint format, nudged elastic band, saddle search, units

Program Summary

Program Title: readcon-core
CPC Library link to program files: (to be added by the Technical Editor)
Developer's repository link: `https://github.com/lode-org/readcon-core`

Email address: `rgoswami@ieee.org` (Rohit Goswami)

Licensing provisions: MIT license (MIT)
Programming language: Rust (core); C99 and C++17 headers; Python 3.10+; Julia; Fortran 2008 (`ISO_C_BINDING`)
Supplementary material: specification `docs/orgmode/spec.org`; clause index `docs/orgmode/clauses.tsv`; Phase A corpus `resources/conformance/`; benchmark scripts `paper/cpc/bench/`; HCN and HNC endpoints `paper/cpc/data/`; printers `scripts/print_neb_band.py`, `examples/print_neb_band.cpp`, `examples/print_neb_banc`
Nature of problem: Saddle searches, adaptive kinetic Monte Carlo, and nudged elastic band calculations store a geometry after many potential calls. The checkpoint has to carry the cell, the masses, a constraint on each Cartesian axis, the atom order from before the file groups atoms by type, and, when a trajectory is restarted, the velocities in a declared time unit. eOn writes this state as CON text. Independent readers do not agree on column 4. The ASE eon reader treats a nonzero flag as a fully fixed atom and does not keep column 5. A force section after the coordinates does not parse in that reader. Files from specification 3.0, including those written by eOn 3.5 and later through this library, can carry natural-unit velocities under a time label of fs. A reader that trusts the label fs scales the kinetic energy by the square of the time-unit factor. A campaign that mixes eOn, ASE, and readcon-core needs one reading of the mask, the id, and the time unit.
Solution method: Specification 3.1 is the contract, and the normative text is `docs/orgmode/spec.org`. Line 2 is one JSON object. `con_spec_version` is the major version, written as a JSON number, and `con_spec_minor` is the minor. Column 4 is an integer flag. In spec 1 the value 1 means fixed on all three axes, and from spec 2 the value is a bitmask with bit 0 for x, bit 1 for y, and bit 2 for z. Column 5 is an integer atom id from 0 through $2^{64}-1$, parsed as an integer. From minor 1, a velocities section requires `units.time`, and the natural unit is written `(amu*angstrom^2/eV)^0.5`. readcon-core is a Rust reader and writer for this text. The shipped C header `include/readcon-core.h` is the language-neutral surface. Python, Julia, and Fortran call that surface, and the Python extension can also call the Rust parser. Text CON remains the file that other codes have to read. gzip and zstd are compression envelopes around that text. *Additional comments including restrictions and unusual features:* The crate on this tree is 0.16.2. The C ABI is not described as a frozen 0.14 series. `rkr_frame_builder_build` returns a null pointer when the builder rejects the frame, and this tree has no `rkr_last_error`, so the reason does not reach C, Fortran, or Julia. The default writer prints six digits after the decimal point. A binary64 round-trip mode exists as

`FloatFormat::RoundTrip` and is not the default. The gzip reader returns the first member of a multi-member file, while the specification requires every member. The Python extension uses pyo3 0.28 with the extension-module feature and is not a free-threaded 3.13t or 3.14t build. Cite the software through `CITATION.cff` and the concept DOI `10.5281/zenodo.18753567`. *References:* R. Goswami, MethodsX 2026, doi:10.1016/j.mex.2026.103899; R. Goswami, MethodsX 2026, doi:10.1016/j.mex.2026.103851; R. Goswami et al., J. Chem. Theory Comput. 2025, doi:10.1021/acs.jctc.5c00866; S. T. Chill et al., Modelling Simul. Mater. Sci. Eng. 2014, doi:10.1088/0965-0393/22/5/055002.

1. Introduction

Saddle searches, adaptive kinetic Monte Carlo, and the nudged elastic band build a path or a rate from many atomic geometries. Each geometry is a checkpoint: the cell, the masses, which Cartesian axes are held fixed, the order the atoms had before they were grouped by chemical type, and sometimes the velocities. CON is the checkpoint used by eOn in those workflows [1, 2, 3, 4]. A surface atom that is fixed in one direction and free in the others cannot be described by a single boolean. Type grouping reorders the atoms, so an image of a path cannot be compared with the next image unless the pre-grouping id is kept. Velocities are useless for a restart if two readers apply different time units to the same numbers. Historical CON readers treat column 4 as a boolean, drop column 5, and have no units object on a version 1 file [5, 4]. Extended XYZ can name extra columns on a comment line, and it does not define this mask or this id [5]. H5MD can store positions, velocities, forces, and unit attributes, and it has no per-axis freeze mask [6]. A LAMMPS dump can store ids and forces, and it has no specification version for a three-bit mask [7]. chemfiles reads many trajectory formats and does not read CON [8]. The OVITO 3.15.4 input list and the Atomsk formats table likewise have no CON reader [9, 10]. This paper states specification 3.1, compares that text with readcon-core 0.16.2, and runs one public path through the file. The significance for a campaign is that the mask, the id, and the time unit then have one reading.

2. The CON format v3.1

The normative text is `docs/orgmode/spec.org`, dated 2026-10-11, major 3 and minor 1. This section is a summary. Where this summary and

the specification disagree, the specification wins. A file is a concatenation of frames. The extensions are `.con`, `.convel`, and the envelopes `.con.gz`, `.convel.gz`, `.con.zst`, and `.convel.zst`. Figure 1 shows one frame and the fields a spec 1 reader does not use.

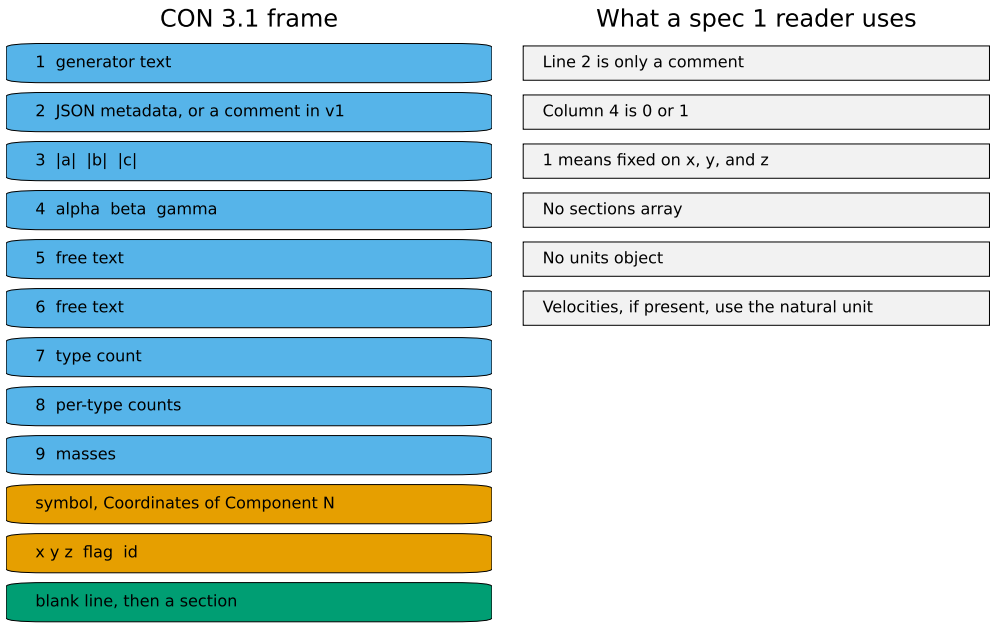


Figure 1: One CON frame, and the fields visible to a spec 1 reader.

Line 2 of a version 2 or 3 frame is one UTF-8 JSON object, with no byte-order mark. `con_spec_version` is a JSON number, the major version. A missing `con_spec_minor`, or an explicit 0, means minor 0. Readers accept unknown minors of a known major and reject an unknown major. JSON with major 0 or 1 is invalid (CON-line2.5). Unknown line-2 keys are preserved on rewrite (CON-line2.8, D14, D15, D17). Line 3 is the lengths of a, b, and c. Line 4 is alpha, beta, and gamma in degrees, with alpha the angle between b and c, beta the angle between a and c, and gamma the angle between a and b. The vector a lies along x and b lies in the xy plane (CON-cell.6, D12). When `lattice_vectors` is present, a reader prefers those three row vectors, and a writer keeps lines 3 and 4 consistent with them at the precision it is using (CON-cell.1, CON-cell.2). A missing `pbc` key means periodic in all three directions (CON-cell.3). A type count of 0 is a legal zero-atom frame:

lines 8 and 9 are empty, no coordinate or section block follows, and the legacy velocity fallback does not apply (CON-count.1, D4). When the type count is at least 1, each per-type count is at least 1 (CON-count.3). Column 4 is an unsigned integer token. In spec 1, 0 is free and 1 is fixed on all three axes. From spec 2, values 0 through 7 are a bitmask, bit 0 for x, bit 1 for y, and bit 2 for z, and a writer emits 7 for an atom fixed on all three axes (CON-mask.1 through CON-mask.7, D8). A float spelling, a sign, or any other value is `InvalidFixedFlag` in every mode (CON-mask.5). Column 5 is an integer id from 0 through $2^{64} - 1$, parsed as an integer and not through binary64 (CON-id.1, D9). No base is implied. Writers emit ids that are unique within the frame. Readers accept repeated ids and keep file order in that case. When the ids are unique, ascending id is the pre-grouping order. A line with four columns takes the 0-based position of the atom in the frame (CON-id.9). A section is present exactly when its name is in `sections` (CON-section.1, D3). Exactly one blank line precedes each section block, and the component blocks inside a section are not separated by a blank line (CON-section.2, CON-section.3, D5). The per-atom energy section is named `energies` (CON-section.5, D6). An unknown section name is an error when the frame minor is not newer than the reader. A newer minor may skip an unknown section whose value-field count is given by `section_arity` (CON-section.7, CON-section.8, D14). `bonds` is optional topology on major 3 and is not a section (CON-bonds.1, D7). Version 3 requires `units.length` and `units.energy`. Velocity is length divided by time. Versions 1, 2, and 3.0 read velocities in the natural unit, and a `units.time` label on those versions is not trusted (CON-units.8, CON-units.9, D10). From minor 1, `units.time` is authoritative, a velocities section requires it, and the natural unit is written exactly `(amu*angstrom^2/eV)^0.5` (CON-units.3, CON-units.5, CON-units.10). A writer that is given an invalid unit fails with `InvalidUnits` and does not substitute a default (CON-units.13, CON-units.14, D11). Readers reject NaN, Inf, and the spellings of those words in every numeric field and in every mode, as `NonFiniteValue` (CON-num.1, D1). `validate` true adds checks and does not relax that rule (CON-validate.2, D1). The C ABI uses a NaN return for some absent numeric getters. That NaN is an API sentinel, it is documented in `docs/orgmode/implementation-notes.org`, and it is not written to a file (CON-num.4). A conforming writer in its default mode writes each finite float so that it parses back to the same binary64 (CON-num.6, D13). Fixed-decimal text is allowed only when the caller asks for it. Trailing whitespace after the last complete frame is ignored. Any other par-

tial frame is `IncompleteHeader`, `IncompleteFrame`, or `IncompleteSection` (CON-frame.1 through CON-frame.6, D2). A blank line followed only by whitespace to end of file is not a legacy velocity block (CON-frame.5). gzip exposes every member of a multi-member file, and zstd exposes every frame (CON-compress.1, CON-compress.2, D16). Labels are compared without regard to case after runs of spaces are collapsed (CON-file.5, D17).

3. Conformance and compatibility

Table 1 counts the clause index emitted from the specification. The index on this tree has 96 MUST clauses, 19 MUST NOT clauses, 2 SHOULD clauses, 1 SHOULD NOT clauses, and 6 MAY clauses.

Table 1: Clause index of CON specification 3.1. Source: `docs/orgmode/clauses.tsv`, copied to `paper/cpc/data/clause_counts.csv`.

Keyword	Count
MAY	6
MUST	96
MUST NOT	19
SHOULD	2
SHOULD NOT	1

The Phase A corpus under `resources/conformance/` has 7 valid fixtures and 7 invalid fixtures (Table 2). `tests/conformance_corpus.rs` and the Python, C, Fortran, and Julia golden tests read that manifest. `tests/spec_v31.rs` checks that the clause index matches `spec.org`, and it parses the specification's minor-1 example, the same bytes read as spec 1, and the zero-atom example. That file does not reject a partial frame, a non-finite token, a bad column-4 spelling, or a second gzip member. Those checks are not in this tree. The manifest entry `column-4-out-of-range` records `ValidationError` and its note says `validate` true rejects the value 8. The specification requires `InvalidFixedFlag` in every mode, including when `validate` is absent (CON-mask.5). The Phase A corpus is therefore not a T1 lock of specification 3.1.

Table 3 and Table 4 compare the fields that the checkpoint needs. Figure 2 is the same codes. Each cell is sourced in Table 5. readcon-core 0.16.2 keeps a declared forces section and the spec major. Its mask and id paths are partial because the lax reader parses those columns through binary64.

Table 2: Phase A corpus on this tree. Source: `resources/conformance/manifest.toml`, copied to `paper/cpc/data/phase_a.csv`. This corpus is not the v3.1 T1 set.

Id	Recorded error
`v2-minimal`	valid
`v3-units`	valid
`column-4-spec2-x-only`	valid
`column-4-spec1-all-fixed`	valid
`column-4-spec2-all-fixed`	valid
`column-4-spec2-y-only`	valid
`column-4-spec2-xz`	valid
`missing-spec-version`	MissingSpecVersion
`v3-missing-units`	ValidationError
`v3-bad-units`	ValidationError
`column-4-out-of-range`	ValidationError
`unsupported-spec-version`	UnsupportedSpecVersion
`unknown-section`	UnknownSection
`validate-true-missing-sections`	InvalidMetadataJson

Its velocity time unit and its gzip reader do not implement CON-units.10 and CON-compress.1. ASE 3.29.0 `ase/io/eon.py` drops the per-axis mask, the pre-grouping id, and a following section. extXYZ, H5MD, and LAMMPS dump can carry related columns or groups, and none of them is this CON contract [5, 6, 7]. chemfiles, OVITO 3.15.4, and the Atomsk formats table have no CON reader [8, 9, 10].

Table 3: Feature codes for readcon-core 0.16.2 and five other formats. Each cell is sourced in Table 5 and in `paper/cpc/data/features.csv`.

Feature	readcon-core 0.16.2	ASE eon.py 3.29.0	extXYZ	H5MD	LAMMPS dump	chemfiles
Per-axis mask	partial	drops	absent	absent	absent	absent
Pre-grouping id	partial	drops	partial	partial	partial	absent
Velocity time unit	open	absent	absent	partial	absent	absent
Declared forces	keeps	drops	partial	partial	partial	absent
Spec version	keeps	drops	absent	absent	absent	absent
Multi-member gzip	partial	absent	absent	absent	absent	absent

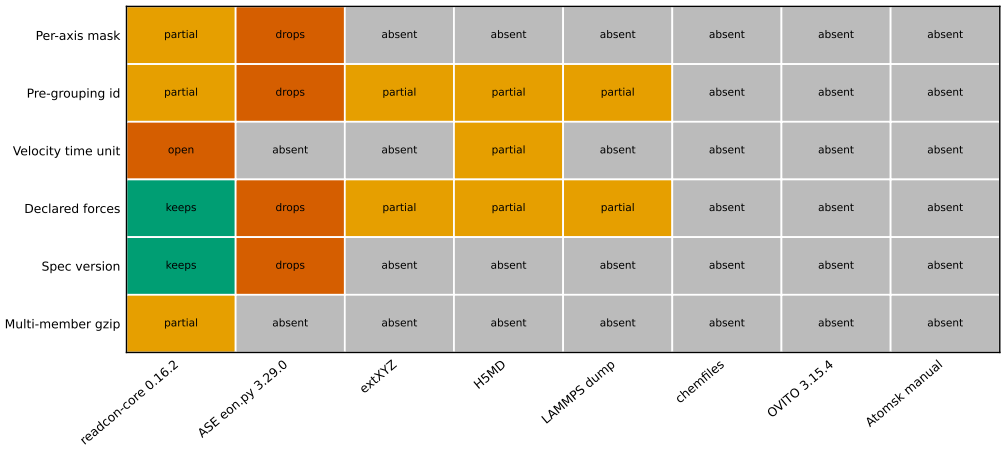


Figure 2: Compatibility codes. The sources are Table 5.

Table 4: OVITO and Atomsk have no CON reader in the manuals cited in Table 5.

Feature	readcon-core 0.16.2	OVITO 3.15.4	Atomsk manual
Per-axis mask	partial	absent	absent
Pre-grouping id	partial	absent	absent
Velocity time unit	open	absent	absent
Declared forces	keeps	absent	absent
Spec version	keeps	absent	absent
Multi-member gzip	partial	absent	absent

Table 5: Source of each comparison cell. The same rows are `paper/cpc/data/features.csv`.

Feature	Reader	Source
Per-axis mask	readcon-core 0.16.2	src/parser.rs:668-676; src/types.rs:644-649
Per-axis mask	ASE eon.py 3.29.0	ase/io/eon.py:139, cited in docs/orgmode/spec.org
Per-axis mask	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e; ASE 3.29 ase/io/extxyz.py
Per-axis mask	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018

Feature	Reader	Source
Per-axis mask	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Per-axis mask	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207; docs/orgmode/spec.org compatibility table
Per-axis mask	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Per-axis mask	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11; Hirel 2015, doi:10.1016/j.cpc.2015.07.012
Pre-grouping id	readcon-core 0.16.2	src/parser.rs:665-675
Pre-grouping id	ASE eon.py 3.29.0	ase/io/eon.py:138-139, cited in docs/orgmode/spec.org
Pre-grouping id	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e
Pre-grouping id	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018
Pre-grouping id	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Pre-grouping id	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207
Pre-grouping id	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Pre-grouping id	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11
Velocity time unit	readcon-core 0.16.2	src/parser.rs:900; src/writer.rs:269-277; src/units.rs:439-445
Velocity time unit	ASE eon.py 3.29.0	ase/io/eon.py:303, cited in docs/orgmode/spec.org
Velocity time unit	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e

Feature	Reader	Source
Velocity time unit	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018
Velocity time unit	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Velocity time unit	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207
Velocity time unit	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Velocity time unit	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11
Declared forces	readcon-core 0.16.2	src/parser.rs:966
Declared forces	ASE eon.py 3.29.0	ase/io/eon.py:177, cited in docs/orgmode/spec.org
Declared forces	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e
Declared forces	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018
Declared forces	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Declared forces	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207
Declared forces	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Declared forces	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11
Spec version	readcon-core 0.16.2	src/parser.rs; tests/spec_v31.rs v31_example_parses
Spec version	ASE eon.py 3.29.0	ase/io/eon.py:72, cited in docs/orgmode/spec.org
Spec version	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e
Spec version	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018

Feature	Reader	Source
Spec version	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Spec version	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207
Spec version	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Spec version	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11
Multi-member gzip	readcon-core 0.16.2	src/compression.rs:82-88
Multi-member gzip	ASE eon.py 3.29.0	ase/io/eon.py:177, cited in docs/orgmode/spec.org
Multi-member gzip	extXYZ	Larsen et al. 2017, doi:10.1088/1361-648X/aa680e
Multi-member gzip	H5MD	de Buyl et al. 2014, doi:10.1016/j.cpc.2014.01.018
Multi-member gzip	LAMMPS dump	Thompson et al. 2022, doi:10.1016/j.cpc.2021.108171
Multi-member gzip	chemfiles	Fraux et al. 2023, doi:10.5281/zenodo.1202207
Multi-member gzip	OVITO 3.15.4	https://www.ovito.org/docs/current/python/modules/ovito_i retrieved 2026-10-11
Multi-member gzip	Atomsk manual	https://atomsk.univ-lille.fr/doc/en/formats.html retrieved 2026-10-11

4. Implementation

The crate on this tree is readcon-core 0.16.2. The Rust types `ConFrame` and `ConFrameBuilder` own the frame. The hot path is the hand-written parser in `src/parser.rs`. The optional Pest grammar in `grammar/readcon.pest` is not used on that path. That grammar accepts a signed integer in column 4. The lax parser reads column 4 and column 5 as binary64 and then casts them (`src/parser.rs:668-676`). With `validate` true, column 4 is parsed as an integer and values above 7 become `ValidationError`, which is not the `InvalidFixedFlag` result the specification names for every mode. The error enum in `src/error.rs` has no `NonFiniteValue`, `InvalidFixedFlag`, `InvalidCount`, or `InvalidUnits` variant. `src/ffi.rs` generates the shipped header `include/readcon-core.h`. C++ uses that header. Python, Julia, and Fortran call the C ABI, and the Python extension can also call the Rust parser. `rkr_frame_builder_build` returns a null pointer when `ConFrameBuilder::build` returns an error, and it discards the error (`src/ffi.rs:3371-3373`). This tree does not define `rkr_last_error`. Fortran and Julia therefore cannot read a builder failure. Several numeric getters return an IEEE NaN when the value is absent. That sentinel is the C API described in `docs/orgmode/implementation-notes.org`. The writer default is six digits after the decimal point (`src/writer.rs` `DEFAULT_FLOAT_PRECISION`). `FloatFormat::RoundTrip` is available when the caller selects it, and `Default` for `FloatFormat` is the six-digit mode. For a major-3 frame whose units are missing or fail validation, the writer inserts `units` with `time` set to `fs` (`src/writer.rs:269-277` and `src/units.rs:439-445`). The specification forbids that substitution (CON-units.14). `src/units.rs` parses the natural spelling `(amu*angstrom^2/eV)^0.5` as a time. `canonicalize_unit_expression` rewrites that spelling with spaces, so a rewrite does not keep the pinned characters (CON-units.5). The gzip path uses `GzDecoder`, which stops after the first member (`src/compression.rs:82-88`). zstd is an optional Cargo feature and is not part of a default build. `read_all_frames` parses sequentially unless the crate is built with `parallel` and the buffer is at least 48 KiB. `read_all_frames_with_threads` takes `Some(1)` for one thread and `Some(n)` for a pool of `n` workers when `n` is at least 2. The frame boundaries are computed before that pool runs, so the partition depends on the input and not on the worker count. The Python package enables `parallel`. Its pyo3 dependency is version 0.28 with the feature `extension-module`. The manifest does not select a free-threaded build, and this paper does not claim a 3.13t or 3.14t wheel. `to_ase` builds the cell from lines 3 and 4 and passes `pbc`

true (`src/python.rs:1498-1516`). It does not read `lattice_vectors`, and it does not read a `pbc` array from the metadata. readcon-db is a separate LMDB index of energy, formula, and section presence over the same CON text. It is not a second program in this paper. A Cap'n Proto schema in the repository is an in-process RPC encoding. It is not the on-disk CON file.

5. Validation

The checks that ship on this tree are the Phase A goldens, `tests/spec_v31.rs`, and writer round trips in `tests/roundtrip_writer.rs` and `tests/ffi_writer_roundtrip.rs`. This tree has no property-test suite and no fuzzer. A T1 corpus that locks the 3.1 error classes is not merged here. Table 6 is the velocity-unit example. eOn writes `.convel` velocities in the natural unit and labels the time column with the constant 10.1805055 fs (`include/eon/ParametersOptions.h:26`). readcon-core 0.16.2 parses the pinned spelling to the factor in the table. The relative difference between those two factors is the relative-difference row. The kinetic energies are the measurements recorded with the specification for one 336-atom Pt slab at 300 K. They are not recomputed in this archive. Reading the natural-unit numbers as angstrom per fs multiplies the kinetic energy by the square of the eOn factor. Six digits after the decimal, which is eOn's `kConvelPrecision` (`client/ConFileIO.cpp:43`) and also the readcon-core default, changes that kinetic energy by the relative loss in the table. Version 3.0 files that carry `time:  fs` while storing natural-unit numbers are the files this factor applies to (CON-units.9). From minor 1 the label has to be the unit the numbers are in (CON-units.10).

Table 6: Velocity unit check. The kinetic energies are the measurements named in the specification. The factor is computed on this tree. Source: `paper/cpc/bench/results/velocity_case.csv`.

Quantity	Value
readcon natural unit (fs)	10.1805057108
canonical spelling	(amu * angstromˆ2 / eV) ˆ0.5
eOn timeUnit (fs)	10.1805055
relative difference	2.07e-08
KE in the natural unit (eV)	12.386
KE if those numbers are read as fs (eV)	1283.72
six-decimal relative KE loss	5.8e-7

6. Performance

The timings below were measured with `paper/cpc/bench/run_bench.py` on the host recorded in `paper/cpc/bench/results/repeats.csv`. readcon-core runs in one long-lived process started from `paper/cpc/bench/readcon-bench`. Each peer runs in the Python process. The paired baseline passes thread count 0, which selects the library default. The library sources are the sources of the base SHA in that CSV. The commit column is the git revision that contains these scripts. Each cell is 30 repeats in the order baseline, peer, baseline, peer. The ratio is the readcon-core time divided by the peer time. The interval is a BCa 95 percent interval of the median of that ratio. A timeout remains in the raw CSV and is not a time. A reader that raises remains in the failure column and is not a time. The payloads are 1, 100, and 10000 frames of the 218-atom file `resources/test/cuh2.con`, one frame of 10000 atoms built by tiling that file, and the sectioned file `resources/examples/neb_band.con`. ASE is `ase.io.read` with `format`"eon"=. chemfiles reads a lean XYZ file of the same coordinates. MDAnalysis and h5py read an H5MD file of the same coordinates [11, 12]. On the sectioned file those three peers are given the CON path, so a failure there is a failure to read CON. Cachegrind instruction counts are one whole-process run of the 1-frame payload, with cache simulation turned off. Figure 3 shows the intervals that exist. Figure 4 shows speedup and efficiency at 1, 2, 4, and 8 threads on the 10000-frame file. The field hash at 1, 2, 3, 4, and 8 threads is the hash column of Table 8.

Holm-adjusted intervals on the readcon-core to peer time ratio lie entirely below 1 for atoms-10000 / ase_eon, atoms-10000 / chemfiles_xyz, atoms-10000 / mda_h5md, frames-1 / ase_eon, frames-1 / chemfiles_xyz, frames-1 / h5py, frames-1 / mda_h5md, frames-100 / ase_eon, frames-100 / chemfiles_xyz, frames-100 / mda_h5md, frames-10000 / ase_eon, frames-10000 / chemfiles_xyz, frames-10000 / mda_h5md.

Holm adjustment is applied to the cells that have a ratio, across that payload and peer grid. 13 of those cells are marked faster for readcon-core in `ratios.csv`. The sentence above is that result.

7. Application

The public campaign is the HCN to HNC isomerization used to test an eOn nudged elastic band workflow [1]. The endpoint files are `examples/hcn_isom/hcn.con`

Table 7: Paired ratios on this host. The ratio is readcon-core time divided by the peer time. The interval is a BCa 95 percent interval of the median. Timeouts and reader failures are counts, not times. Source: `paper/cpc/bench/results/ratios.csv`.

Payload	Peer	OK	Fail	Timeout	Ratio	BCa interval
atoms-10000	ase_eon	30	0	0	0.0857228	[0.084647, 0.0878791]
atoms-10000	chemfiles_xyz	30	0	0	0.386266	[0.383483, 0.395421]
atoms-10000	h5py	30	0	0	2.57362	[2.54415, 2.68472]
atoms-10000	mda_h5md	30	0	0	0.185294	[0.182484, 0.188263]
frames-1	ase_eon	30	0	0	0.1101	[0.105751, 0.114059]
frames-1	chemfiles_xyz	30	0	0	0.561037	[0.548471, 0.56682]
frames-1	h5py	30	0	0	0.162082	[0.147114, 0.172554]
frames-1	mda_h5md	30	0	0	0.0103318	[0.0099818, 0.0105204]
frames-100	ase_eon	30	0	0	0.0287282	[0.027767, 0.0303464]
frames-100	chemfiles_xyz	30	0	0	0.158339	[0.154832, 0.159886]
frames-100	h5py	30	0	0	2.20048	[2.10544, 2.31128]
frames-100	mda_h5md	30	0	0	0.015218	[0.0145882, 0.0159728]
frames-10000	ase_eon	30	0	0	0.0312095	[0.0303026, 0.0346157]
frames-10000	chemfiles_xyz	30	0	0	0.166657	[0.158872, 0.169841]
frames-10000	h5py	30	0	0	4.73463	[4.45296, 4.81672]
frames-10000	mda_h5md	30	0	0	0.0182179	[0.0175241, 0.0205931]
sectioned-neb	ase_eon	0	30	0		
sectioned-neb	chemfiles_xyz	0	30	0		
sectioned-neb	h5py	0	30	0		
sectioned-neb	mda_h5md	0	30	0		

and `examples/hcn_isom/hnc.con` at commit `669a58ef` of `eon_orchestrator` [13]. Copies are `paper/cpc/data/hcn.con` and `paper/cpc/data/hnc.con`. Those files are spec 1 text written by ASE: column 4 is 0, the ids are 0, 1, and 2, and there is no velocity block. `paper/cpc/scripts/hcn_case.py` rewrites them as specification 3.1 with `units.time` set to the natural spelling and a velocities section. A third frame keeps the HCN coordinates, sets column 4 of H to 1, and stores the ids 2, 0, and 1 in file order. Under spec 2 and later, column 4 equal to 1 is fixed on x only (CON-mask.4). The published endpoints themselves use column 4 equal to 0. The third frame is a reader probe on those coordinates, not a claim that the published path fixed an axis. Table 10 is the result. readcon-core 0.16.2 returns the masks, the ids, and the velocity numbers, and it keeps the time string on the metadata. It does not convert the velocities into femtoseconds. ASE `format`"eon"= fails on the file that contains the velocities section. On the coordinates-only probe it returns an all-fixed constraint on H and does not return column 5. No eOn binary was run in this archive. eOn 3.6.0 reads masks through `getFixedMask` (`client/ConFileIO.cpp:386`) and writes velocities at six decimals in the natural unit. eOn 2.17.10 collapses any fixed bit to 1

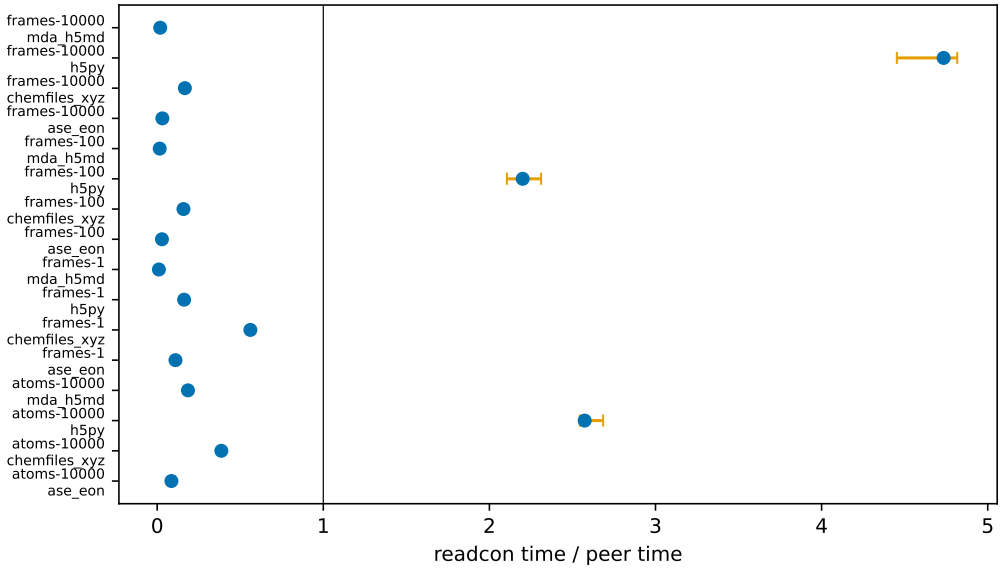


Figure 3: BCa intervals for the ratio of readcon-core time to peer time. A ratio below 1 means a shorter readcon-core time.

(`client/ConFileIO.cpp:384-385`).

The same repository ships a short printer for a four-bead file, `resources/examples/neb_band.c` `scripts/print_neb_band.py`, `examples/print_neb_band.cpp`, and `examples/print_neb_band.f9` print the bead index, the energy, and `fmax` from line 2. The Julia binding exposes `neb_bead` and `neb_band` through the C ABI and does not contain a fourth printer program. Table 11 is the declared metadata. The energies are the line-2 values. They are not a production barrier. The declared difference between bead 1 and bead 0 is 0.80 eV.

Two-dimensional projections of paths that already live in CON files are a separate method and are not recomputed here [2]. Gaussian-process saddle searches that call eOn are reported elsewhere [3].

8. Availability

Source, the C header, and the language bindings are in `https://github.com/lode-org/readcon-core`. The crate version on this tree is 0.16.2. Python wheels are published as `readcon` and `readcon-chemfiles`. Julia and Fortran bind `libreadcon_core` from this repository and are not separate registry re-

Table 8: Thread scaling on 10000 frames of `cuh2.con`. Speedup uses the 1-thread median. Source: `paper/cpc/bench/results/threads_summary.csv`.

Threads	Median (s)	Speedup	Efficiency	Field hash
1	0.279674	1	1	yes
2	0.189806	1.47347	0.736736	yes
4	0.12536	2.23096	0.55774	yes
8	0.0966242	2.89445	0.361806	yes

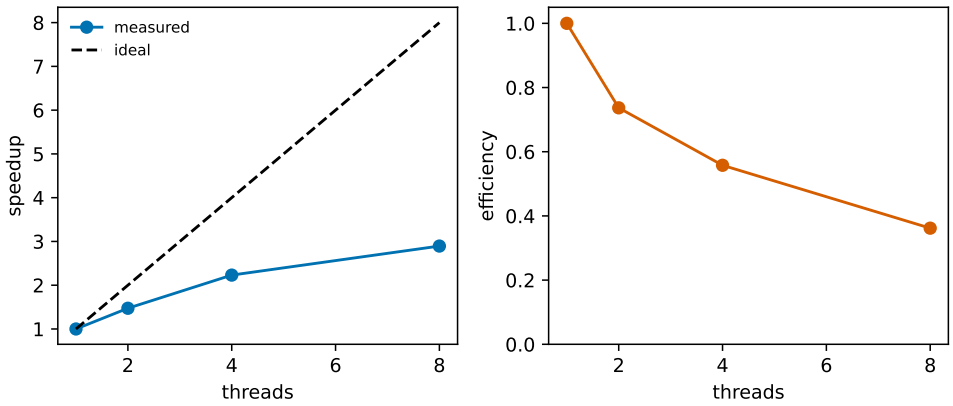


Figure 4: Speedup and efficiency against one thread, on 10000 frames of `cuh2.con`.

leases in this paper. CMake and Meson consume the cxx source tarball attached to a GitHub release. Cite the software with `CITATION.cff` [14, 15].

Data statement

The software archive is the Zenodo concept DOI 10.5281/zenodo.18753567, and the program files will also be deposited in the CPC Program Library. The benchmark CSVs and the scripts that wrote them are in this archive under `paper/cpc/bench/` and `paper/cpc/data/`. The HCN and HNC endpoints are the public files of `https://doi.org/10.1016/j.mex.2026.103899` at the commit named above [1, 13].

Table 9: Cachegrind instruction references for one whole-process parse of the 1-frame payload. Source: `paper/cpc/bench/results/cachegrind.csv`.

Payload	I-refs	Status
frames-1	824 686	ok

Table 10: Readers on the v3.1 rewrite of the public HCN/HNC endpoints. Source: `paper/cpc/bench/results/hcn_loss.csv`.

Reader	File	Status	Record
readcon-core 0.16.2	`hcn_v31.con`	ok	masks 000 000 000; ids 0 1 2
readcon-core 0.16.2	`hcn_v31.con`	ok	masks 000 000 000; ids 0 1 2
readcon-core 0.16.2	`hcn_v31.con`	ok	masks 000 000 100; ids 2 0 1
ASE format=eon	`hcn_v31.con`	failure	ValueError: could not convert string to float: 'Velocities'
ASE format=eon	`hcn_v31_coords.con`	ok	fixed=free,free,all-fixed;id=False;velocity=False;symbols=C,N,H
eOn 3.6.0	`hcn_v31.con`	not-run	No eOn binary was executed. client/ConFileIO.cpp:386 getFixedMask and :714 setFixedMask keep a mask. v2.17.10 client/ConFileIO.cpp:384-385 collapses any fixed bit to 1. include/eon/ParametersOptions.h:26 timeUnit is 10.1805055 fs.

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Validation, Writing - original draft, Writing - review & editing.

Declaration of competing interests

The author declares no competing interests.

Funding

Acknowledgements

The author thanks Prof. Birgir Hrafnkelsson, Prof. Thomas Bligaard, Dr. Andreas Vishart, and Dr. Miha Gunde for helpful discussions. R.G. also acknowledges valuable discussions with Dr. Amrita Goswami, Dr. Moritz Sallermann, Prof. Debabrata Goswami, Mrs. Sonaly Goswami, and Mrs. Ruhila Goswami. Financial support from Dr. Guillaume Fraux and Prof.

Table 11: Declared bead metadata in `resources/examples/neb_band.con`. Source: `paper/cpc/data/neb_band.csv`. E and $f_{\max}$ are line-2 JSON.

Bead	Role	E (eV)	$f_{\max}$
0	reactant	-1.20	0.31
1	saddle	-0.40	0.08
2	descending	-0.65	0.12
3	product	-1.10	0.27

Michele Ceriotti of Lab-COSMO, EPFL is gratefully acknowledged. The figure color scheme was designed by Ruhila Goswami. The author thanks his family, pets, plants, birds, and garden creatures for their patience and support throughout this work.

References

[1] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.

[2] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.

[3] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of Gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.

[4] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.

[5] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin,

E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648X/aa680e.

[6] P. de Buyl, P. H. Colberg, F. Höfling, H5MD: A structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.

[7] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.

[8] G. Fraux, L. Kimms, J. Fine, G. P. Barletta, M. Dimura, F. Coudert, pelsa, M. Levesque, S. Maiti, S. Guionniere, jmintser, chemfiles (2023). doi:10.5281/zenodo.1202207.
URL https://doi.org/10.5281/zenodo.1202207

[9] A. Stukowski, Visualization and analysis of atomistic simulation data with OVITO, the open visualization tool, Modelling and Simulation in Materials Science and Engineering 18 (1) (2010) 015012. doi:10.1088/0965-0393/18/1/015012.

[10] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files, Computer Physics Communications 197 (2015) 212–219. doi:10.1016/j.cpc.2015.07.012.

[11] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.

[12] R. Gowers, M. Linke, J. Barnoud, T. Reddy, M. Melo, S. Seyler, J. Domański, D. Dotson, S. Buchoux, I. Kenney, O. Beckstein, MDAnalysis: A Python package for the rapid analysis of molecular dynamics

simulations, in: Proceedings of the 15th Python in Science Conference, 2016, pp. 98–105. doi:10.25080/majora-629e541a-00e.

[13] R. Goswami, eon_orchestrator, commit 669a58efc3b193286e9318cc258127cd65e06a63 (2026).
URL https://github.com/HaoZeke/eon_orchestrator

[14] A. M. Smith, D. S. Katz, K. E. Niemeyer, Software citation principles, PeerJ Computer Science 2 (2016) e86. doi:10.7717/peerj-cs.86.

[15] R. Goswami, readcon-core: a con/convel reader/writer with ffi bindings, concept DOI for the readcon-core software archive. (2026). doi:10.5281/zenodo.18753567.
URL https://doi.org/10.5281/zenodo.18753567